

Recursion

- Recursion is a programming technique where a function calls itself to solve a problem.
- Recursion = solving a problem by breaking it into smaller instances of the same problem
- Base Case
 - Condition to stop recursion
 - Prevents infinite calls
- Recursive Call
 - Function calls itself with reduced input
- Recursion means a function solving a problem by calling itself on a smaller input until it reaches a base case.
- Memory Concept
 - Uses call stack
 - Each call creates a stack frame
 - Memory released on return
- Advantages
 - Simple and clean code
 - Useful for tree/graph problems
 - Natural fit for divide & conquer
- Disadvantages
 - Higher memory usage (stack)
 - Slower than iteration
 - Risk of stack overflow
- When to Use
 - Tree traversal
 - Divide & Conquer (Merge Sort, Quick Sort)
 - Backtracking problems

Types of recursion

1. Direct Recursion

- A function calls itself directly.

```
void fun(int n) {  
    if (n == 0) return;  
    printf("%d ", n);  
    fun(n - 1);  
}
```

2. Indirect Recursion

- A function calls another function, which eventually calls the first one.
- Indirect recursion is nothing but a state machine where each function represents a state and calls another state.

```

void fun1(int n);

void fun2(int n) {
    if (n <= 0) return;
    printf("%d ", n);
    fun1(n - 1);
}

void fun1(int n) {
    if (n <= 0) return;
    printf("%d ", n);
    fun2(n - 1);
}

```

3. Tail Recursion

- Recursive call is the last operation in the function.

```

void fun(int n) {
    if (n == 0) return;
    printf("%d ", n);
    fun(n - 1);
}

```

4. Head Recursion

- Recursive call happens before any processing.

```

void fun(int n) {
    if (n == 0) return;
    fun(n - 1);
    printf("%d ", n);
}

```

5. Tree Recursion

- Function calls itself multiple times.

```

void fun(int n) {
    if (n == 0) return;
    printf("%d ", n);
    fun(n - 1);
    fun(n - 1);
}

```

Brute Force Approach

- Brute force means solving a problem by trying all possible solutions and picking the correct one.
- Brute force = check every possibility until you find the answer
- No optimization
- No shortcuts
- Just systematic trial and error
- Characteristics
 - Simple to implement
 - Easy to understand
 - Works for all inputs
 - Usually slow (high time complexity)
- Time Complexity - $O(n^2)$, $O(n^3)$, $O(2^n)$ (for recursion problems)
- Inefficient for large inputs, Not scalable, High execution time

Two Pointer Technique

- The two pointer technique is used to solve problems efficiently by using two indices (pointers) to traverse a data structure (usually arrays or strings).
- Use two pointers instead of one to reduce time complexity (often from $O(n^2) \rightarrow O(n)$)
- Common Patterns
 1. Opposite Direction (Left & Right)
 - One pointer at start, one at end
 - Move based on condition
 - Used in:
 - Pair sum (sorted array)
 - Palindrome check
 2. Same Direction (Slow & Fast)
 - Both start at beginning
 - One moves faster than the other
 - Used in:
 - Remove duplicates
 - Floyd's cycle detection
 3. Sliding Window (Dynamic Range)
 - Maintain a window $[i, j]$
 - Expand and shrink window
 - Used in:
 - Subarray sum
 - Longest substring
- Reduces time complexity, Avoids nested loops, Efficient for sorted data
- Use two pointers technique when
 - Array is sorted
 - Need to find pairs/triplets
 - Working with subarrays/substrings
 - Problems involving ranges